

CODE:

```
Welcome 8 puzzle solver code file.py X
C: > Users > Anshika Singh > 8 puzzle solver code file.py > ...
1  import heapq
2
3  # Goal state
4  goal_state = [[1,2,3],
5                [4,5,6],
6                [7,8,0]]
7
8  # Directions for movement
9  moves = [(-1,0),(1,0),(0,-1),(0,1)]
10
11 # Find position of 0
12 def find_zero(state):
13     for i in range(3):
14         for j in range(3):
15             if state[i][j] == 0:
16                 return i, j
17
18 # Heuristic: Manhattan Distance
19 def heuristic(state):
20     distance = 0
21     for i in range(3):
22         for j in range(3):
23             val = state[i][j]
24             if val != 0:
25                 goal_x = (val - 1) // 3
26                 goal_y = (val - 1) % 3
27                 distance += abs(i - goal_x) + abs(j - goal_y)
28     return distance
29
```

```
Welcome 8 puzzle solver code file.py X
C: > Users > Anshika Singh > 8 puzzle solver code file.py > ...
29
30 # Convert state to tuple (for hashing)
31 def to_tuple(state):
32     return tuple(tuple(row) for row in state)
33
34 # A* Algorithm
35 def solve(start):
36     pq = []
37     heapq.heappush(pq, (0 + heuristic(start), 0, start, []))
38     visited = set()
39
40     while pq:
41         f, g, state, path = heapq.heappop(pq)
42
43         if state == goal_state:
44             return path
45
46         visited.add(to_tuple(state))
47
48         x, y = find_zero(state)
49
50         for dx, dy in moves:
51             nx, ny = x + dx, y + dy
52
53             if 0 <= nx < 3 and 0 <= ny < 3:
54                 new_state = [row[:] for row in state]
55                 new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]
56
57                 if to_tuple(new_state) not in visited:
58                     new_path = path + [new_state]
```


